End-to-End Fraud Detection System: From Risk Scoring to Decision Intelligence

DDM501 Final Project — Group 1 (replace with full member names)

April 15, 2026

Contents

1 Introduction

Fraud detection in payment systems is best framed as a decision problem rather than a pure classification task. A model may produce a score, but a real system must decide which transactions to allow, which to route to review, and which to block, subject to operational constraints such as analyst capacity and latency budgets. This report presents an end-to-end machine learning system that transforms transaction feature vectors into tiered decisions, and provides serving, monitoring, and a decision-support user interface.

The core contribution is an explicit separation between (i) *risk scoring* as a ranking signal and (ii) *decision intelligence* as a policy layer that maps scores to actions. The system is implemented with a reproducible training workflow, a FastAPI serving layer with Prometheus instrumentation, a lightweight web dashboard, and a containerized observability stack.

2 Problem Definition & Business Context

The business objective is to reduce expected fraud loss while controlling customer friction and operational cost. Two error types have asymmetric consequences: false negatives correspond to missed fraud and direct monetary loss; false positives correspond to legitimate transactions that are blocked or reviewed, creating friction and potentially lost revenue. The system therefore must operate at a well-justified operating point, not merely maximize an offline metric.

Operationally, the system is designed to allocate scarce interventions. Let $s(x) \in [0, 1]$ denote the model output for transaction features x. The online decision rule maps $s(x)$ into a tier and an action:

$$x \to s(x) \to \text{tier}(s) \to \text{action}(\text{tier}). \tag{1}$$

This mapping is parameterized by thresholds that are chosen as a business policy and versioned as part of the model release.

3 Dataset & Data Characteristics

The project uses the Kaggle Credit Card Fraud dataset (`creditcard.csv`), which contains 284,807 transactions, 30 features (`Time`, `V1..V28`, `Amount`), and a binary label `Class`. The dataset is extremely imbalanced (fraud rate $\approx 0.17\%$), which makes accuracy an unreliable measure of performance and motivates the use of ranking-focused metrics.

The PCA-derived features `V1..V28` limit interpretability; therefore, this report treats explainability artifacts as evidence of model behavior rather than causal explanations. Any fairness assessment is constrained by the absence of protected attributes.

4 ML Pipeline Design

The benchmark workflow is implemented as a deterministic script that produces a complete evidence pack under `artifacts/`, including model binaries, metadata, figures, and tables. The workflow performs: schema checks, exploratory summaries, stratified train/validation/test splitting, training of a baseline model and a candidate model, validation-only model selection, and export of a deployable artifact along with a machine-readable decision policy.

The split is a stratified random split for stability and reproducibility. This is a deliberate simplification for a student project; in production, a time-aware split would generally be required to better approximate deployment drift and delayed labels.

5 Model Evaluation & Selection

Two model families are evaluated: (i) a baseline Logistic Regression pipeline with standardization and class weighting, and (ii) a tuned LightGBM classifier. Performance is assessed using ROC-AUC and Average Precision (AP, often referred to as PR-AUC), alongside operating-point precision and recall.

Table 1: Model comparison at representative operating points (test split). Values are generated by the repository workflow and stored under `artifacts/benchmarks/model_comparison_table.csv`.

Model	Threshold	Precision	Recall	F1	ROC-AUC	AP (PR-AUC)
Logistic Regression	0.99	0.6742	0.8108	0.7362	0.9680	0.7929
LightGBM	0.84	0.8852	0.7297	0.8000	0.8828	0.7161

The deployable model is selected by a validation-only rule that prioritizes validation AP and uses recall at the review operating point as a tie-breaker. The selected model for the current artifact set is Logistic Regression, and the decision thresholds are stored in `artifacts/models/model_info.json`.

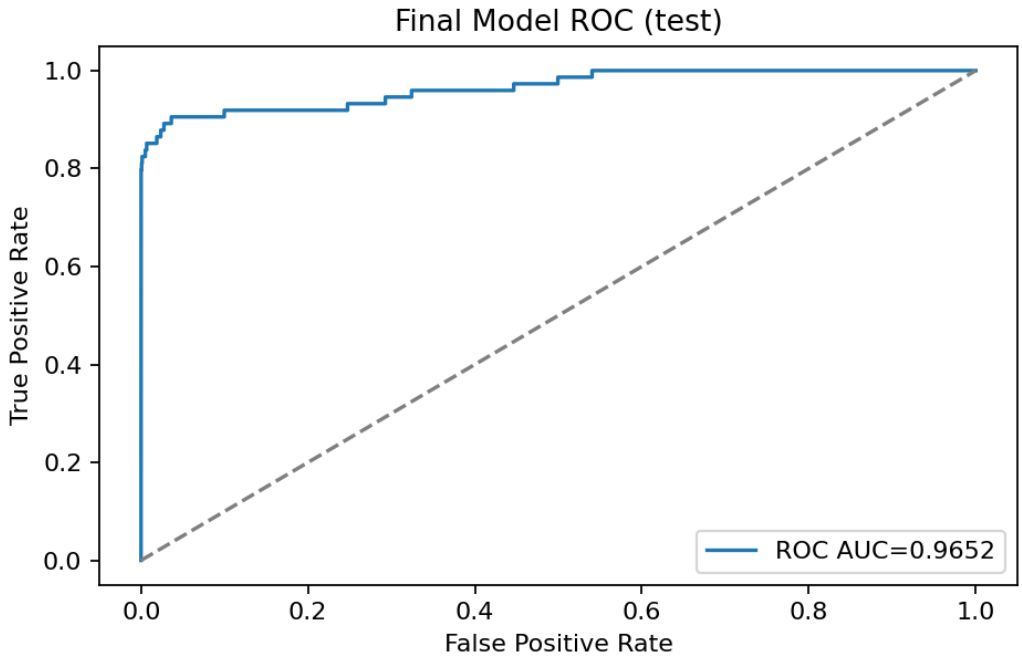

Figure 1: ROC curve for the final model (test split).

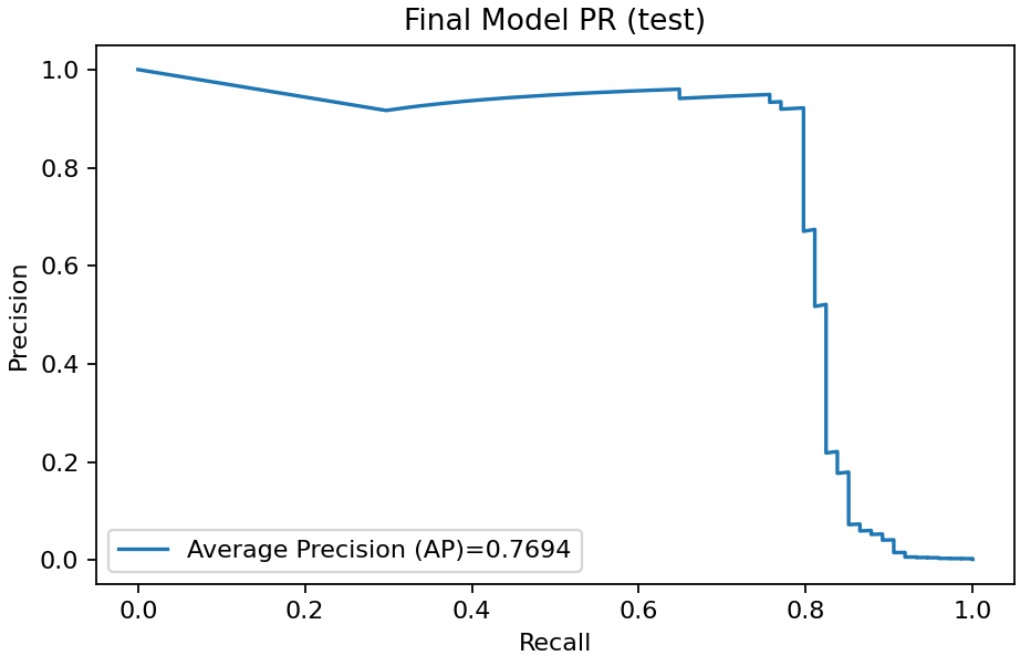

Figure 2: Precision–Recall curve for the final model (test split). The legend reports Average Precision (AP), consistent with exported PR-AUC values.

6 Risk Score Analysis (Calibration Discussion)

6.1 Risk Score Semantics (Not a Probability)

Although the serving API computes the output using `predict_proba`, the returned value is explicitly *not* interpreted as a calibrated probability of fraud. Instead, it is treated as an **uncalibrated risk score** used for ranking and tiering. This is essential for correctness: a score in $[0, 1]$ can be mistaken for a probability, which would lead to invalid business conclusions and miscommunication with stakeholders.

Several factors prevent probability claims in this system: the extreme class imbalance, the use of class weighting, the absence of calibration validation (e.g., calibration curves and Brier score), and the fact that the decision policy is capacity-driven and depends primarily on rank ordering rather than probability accuracy. A production deployment that requires probabilistic interpretation would need explicit post-hoc calibration and monitoring of calibration error under drift.

6.2 Calibration implications for decision-making

Calibration is not required to implement a capacity-driven review policy because top-K thresholding depends on ordering rather than absolute probability. However, calibration becomes important when decisions are cost-based (e.g., expected loss thresholds) or when the score is communicated to humans as a probability. Therefore, this system provides strong semantic warnings and exposes thresholds and tiers as the primary decision interface.

7 Decision Layer (Threshold Strategy)

The system implements a two-tier decision policy based on operational capacity, stored as metadata and returned by the serving API. The policy is defined by two thresholds: a review threshold and a high-risk threshold. Transactions above the high threshold can be blocked (or held), while those between review and high are routed to manual review or step-up authentication. This provides an explicit mechanism to control the review queue size as traffic volume changes.

Table 2: Threshold policy used by the deployed system (from `artifacts/models/model_info.json`). The operating points are capacity-driven, not probability-driven.

Tier	Action	Top-rate target	Threshold
LOW	allow	–	< 0.7391
REVIEW	review	top 1%	≥ 0.7391
HIGH	block	top 0.2%	≥ 0.9999

For completeness, the repository also records an offline comparator threshold that maximizes validation F1 (stored as `threshold_f1` in metadata). This comparator is reported to illustrate that metric-optimal thresholds can differ from capacity-optimal policies; the production-aligned behavior of this system is the top-K tiering policy.

Table 3: Metrics summary for the final model at key operating points (test split). Values are sourced from `artifacts/models/model_info.json`.

Operating point	Threshold	Precision	Recall	F1
Review tier (top 1%)	0.7391	0.1462	0.8514	0.2495
High tier (top 0.2%)	0.9999	0.8429	0.7973	0.8194
F1-optimal (offline)	0.9900	0.7024	0.7973	0.7468

8 System Architecture

The system is organized into layered components: an offline training and benchmarking workflow, an online serving API, a frontend decision interface, and an observability stack. Model artifacts and policy metadata are stored under `artifacts/` and mounted into the serving container at runtime.

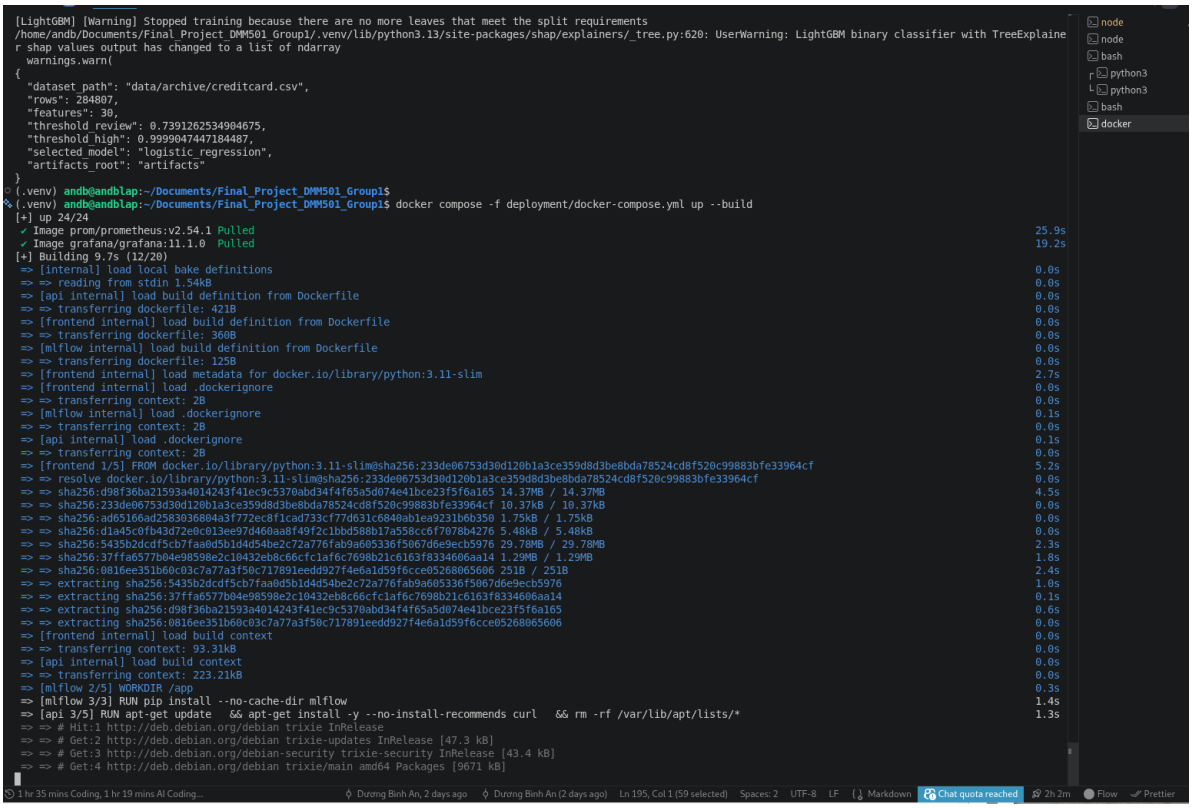

Figure 3: Deployment topology evidence (Docker Compose build and stack start). This screenshot documents the composed services used in the demo environment; a clean architecture diagram is recommended for final submission.

9 API & Serving Layer

The serving layer is implemented using FastAPI and exposes health, prediction, and metrics endpoints. The API validates input feature lengths against the model metadata to prevent silent errors due to contract mismatch. The server exports Prometheus metrics to support latency and error-rate monitoring.

9.1 API example

An example request payload (30 ordered features) and response are shown below. The response includes the tier and action as decision outputs, as well as threshold metadata for transparency.

```
POST /predict
{
  "features": [0.0, -1.2, 0.3, ..., 12.45]
}

200 OK
{
  "request_id": "...",
  "risk_score": 0.812345,
```

```
  "risk_tier": "REVIEW",
  "action": "review",
  "decision_label": "REVIEW",
  "threshold_review": 0.739126,
  "threshold_high": 0.999905,
  "score_semantics": "risk_score_uncalibrated",
  "model_version": "creditcard-production-v1"
}
```


10 Frontend & Decision Interface

The frontend is a lightweight dashboard intended to support a risk-operations workflow. It continuously displays health and policy metadata (model version and thresholds), visualizes a stream of scored events, and maintains a review queue for human handling. Importantly, the UI avoids presenting the score as a probability; it labels it as an uncalibrated risk score and emphasizes tiered decisions.

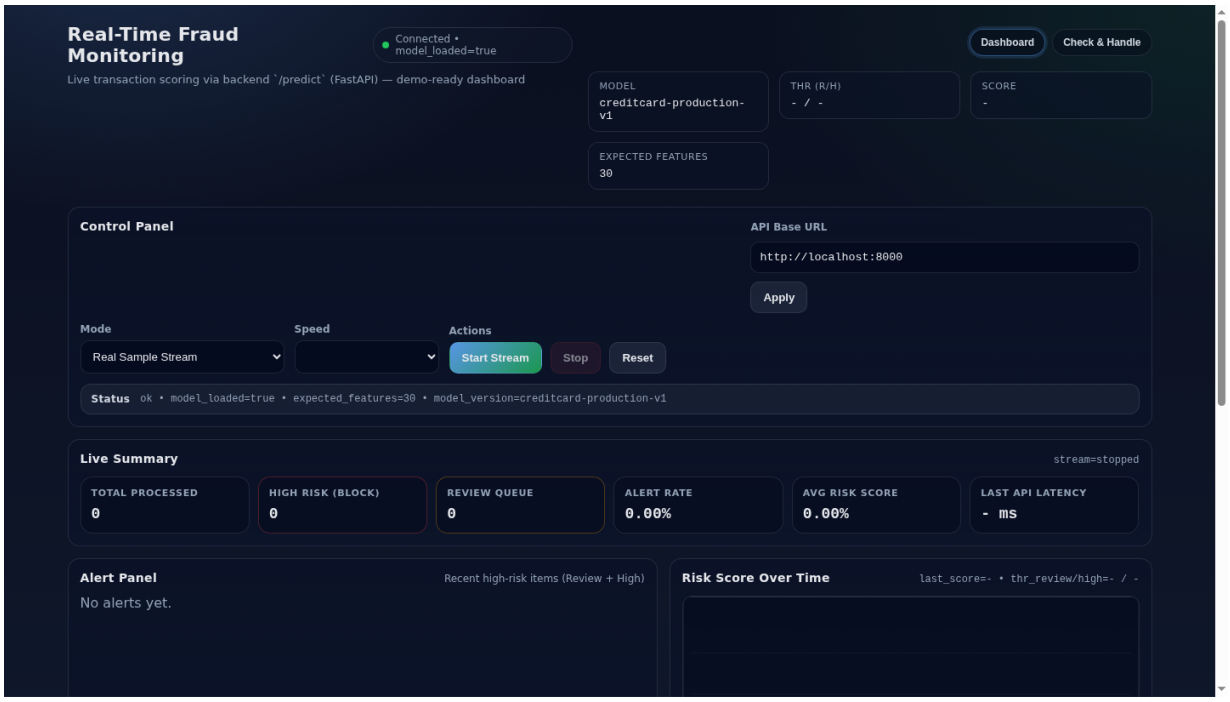

Figure 4: Dashboard UI (demo). The interface displays tiered decisions and includes explicit semantic warnings that the score is not a calibrated probability.

11 Streaming & Simulation

The system includes a pull-based streaming endpoint that returns already-scored events for the UI. The stream generator is a simulation: it produces time-ordered events, supports bursty traffic, and can sample feature vectors from a dataset-backed pool or a synthetic generator. Ground-

truth labels are not returned through the public streaming endpoint, which avoids an unrealistic oracle-style demo.

This streaming layer is explicitly not a production ingestion system. A production architecture would use an event bus, idempotency/deduplication, delayed label ingestion, and data governance around schema evolution and feature provenance.

12 Monitoring & Observability

The API exports Prometheus metrics for request counts, latency histograms, and decision tier distributions. Prometheus scrapes these metrics and evaluates alert rules, while Grafana provides dashboards for operational monitoring. This is necessary for production awareness: without observability, model-serving issues can remain undetected even if offline metrics are strong.

13 Testing & CI/CD

The repository includes unit and integration tests for the API contract, error handling, dataset sampling behavior, and model-loading behavior. Continuous Integration runs the test suite and enforces a coverage gate (80%) using GitHub Actions. Docker build and Compose configuration validation are also automated in CI.

14 Responsible AI

The project documents Responsible AI considerations including fairness limitations, explainability, and privacy. Because the dataset lacks protected attributes, demographic fairness claims cannot be made; instead, the report recommends proxy slice checks (e.g., by amount and time buckets) and treats results as limited evidence. Explainability artifacts are provided via SHAP for candidate models and via coefficient/importance summaries for the deployed model family, with explicit warnings that explanations are not causal.

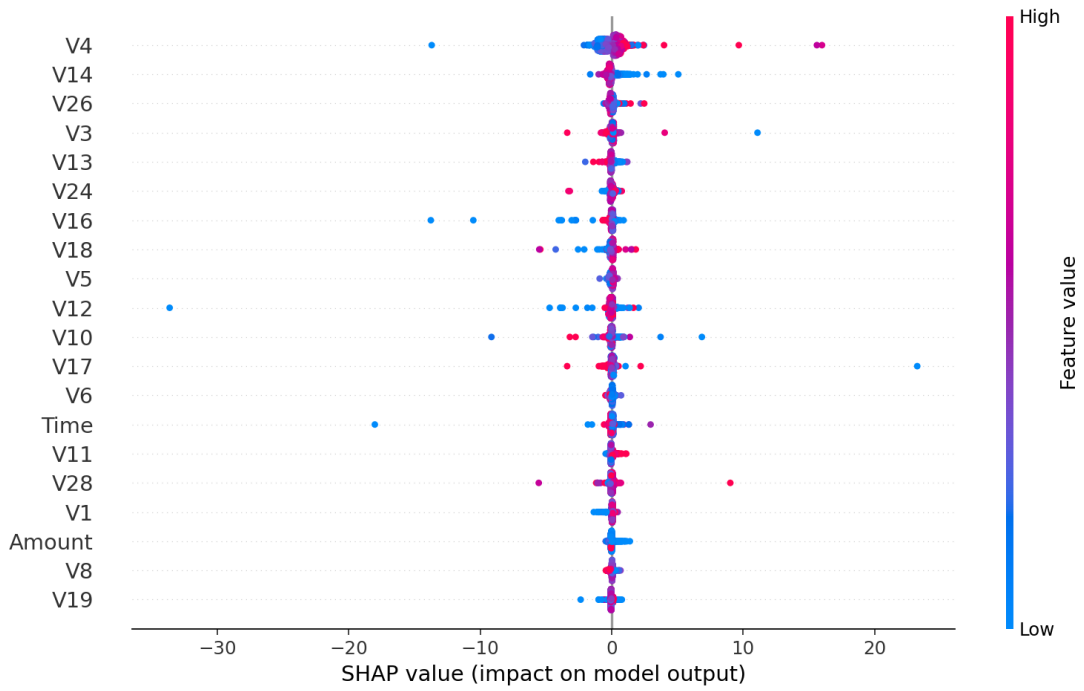

Figure 5: SHAP summary plot (candidate model explainability artifact). Interpret as model behavior evidence, not causal attribution.

15 Limitations & Production Gap

This project is demo-grade rather than production-complete. Key gaps include: absence of authentication and rate limiting, lack of calibrated probabilities and calibration monitoring, use of random splits instead of time-aware validation, simulated streaming rather than event-driven ingestion, and no automated drift detection or retraining pipeline. These limitations are not hidden; they define a clear boundary between a DDM501-capable demonstration and a production fraud platform.

16 Conclusion

This system demonstrates an end-to-end path from data to decisions: model artifacts are trained and versioned, a serving API exposes tiered actions, a frontend dashboard supports review workflows, and an observability stack provides system-level visibility. The system is academically defensible because it distinguishes model ranking from operational policy, and it is industry-aware because it treats monitoring, testing, and deployment as first-class concerns.